

Estimation of Obesity Levels Based on Eating Habits and Physical Condition: A Multi-Class Classification Study

Name: Emma Gogic

Course: CPS 844 - Data Mining

Instructor: Elodie Lugez, Ph.D.

Date: February 20, 2026

1. Motivations and Background

Obesity is a major public-health concern that is associated with increased risk of chronic diseases and reduced quality of life. It is a condition that is being constantly monitored by doctors, researchers and scientists, including data scientists. Data scientists can use data mining to classify the risk of obesity given information about a person's demographic characteristics, eating habits, and physical condition. The results can help us understand which lifestyle factors have the strongest impact on obesity levels.

This report uses the “Estimation of Obesity Levels Based On Eating Habits and Physical Condition” dataset from the UCI Machine Learning Repository. 23% of the data was collected directly from users via a web platform, while 77% was synthetically generated using Weka and SMOTE. The repository describes the dataset as containing 2,111 records and 17 columns (16 predictive attributes and one class label). The target label represents seven obesity categories (NObeyesdad): Insufficient Weight, Normal Weight, Overweight Level I, Overweight Level II, Obesity Type I, Obesity Type II, and Obesity Type III [1]. The numerous categories of obesity means that the data set is multi-class rather than binary. Additionally, the dataset includes both numeric and categorical features (e.g., Age and Weight vs. Gender and transportation mode). These combined aspects make the dataset an interesting classification problem; through the comparison of different classifiers such as linear models, instance-based methods, tree ensembles, etc. we can deduce fascinating results regarding the significance of certain features to the risk of obesity.

2. Methods

2.1. Data and Preprocessing

The dataset was provided as a CSV file with 16 predictor variables and one target variable, “NObeyesdad”, representing seven obesity levels. The data preprocessing was done using Python's scikit-learn library, specifically using the ‘Pipeline’ and ‘ColumnTransformer’ modules. The techniques used for preprocessing varied depending on whether the variables were numeric or categorical. For instance, numeric features were imputed using the median and standardized using z-score normalization, while categorical features were imputed using the most frequent category and one-hot encoding (conversion of categorical

data into a binary matrix). In order to prevent data leakage and ensure fair comparisons between classifiers, all preprocessing steps were embedded inside the modeling pipeline.

2.2. Implementation challenges and fixes

During implementation, two significant technical issues were encountered and resolved. The first issue was:

```
TypeError: LogisticRegression.__init__() got an unexpected keyword
argument 'multi_class'
```

The above error indicated that the 'multi_class' parameter was not recognized. This occurred because in newer versions of scikit-learn (≥ 1.5), the 'multi_class' parameter was deprecated and removed. To resolve this issue and ensure compatibility with the installed scikit-learn version, I simply removed the line, `multi_class="multinomial"`. This did not hinder the functionality of the code, as multinomial behavior is automatically selected.

The second issue that occurred during preprocessing was:

```
ValueError: Cannot use median strategy with non-numeric data: could not
convert string to float: 'Male'
```

This error indicated that at least one categorical column (e.g., Gender) had been incorrectly classified as numeric. As a result, the numeric imputation pipeline attempted to compute the median of string values. To resolve this issue, a robust strategy was implemented to determine whether a category was numeric or categorical. This involved using the 'pd.to_numeric' function from the Python Pandas library to attempt numeric coercion on column values. If $\geq 95\%$ of values were converted successfully, the column would be treated as numeric, otherwise it would be treated as categorical. This ensured that categorical variables such as 'Male' and 'Female' were correctly processed through the categorical pipeline.

2.3. Comparison of Classifiers

Six different classifications methods were trained and evaluated:

1. Multinomial Logistic Regression
2. Linear Support Vector Machine
3. K-Nearest Neighbors (KNN)
4. Decision Tree
5. Random Forest
6. Gradient Boosting

2.4. Evaluation strategy and metrics

To ensure each class is represented proportionally in both sets, A stratified 80/20 train-test split is used for the model comparison. To compare the models, we

examine the accuracy (fraction of correct predictions), Macro-F1 (averages F1 across classes equally) and Weighted-F1 (averages F1 weighted by class support). Then, the script saves a detailed classification report and confusion matrix based on the best model that was determined by Weighted-F1. Additionally, to produce a more stable estimate, there is the option in the script to run a 10-fold stratified cross-validation.

2.5. Feature importance

The importance of any given feature varies depending on the classification model used. In this study, permutation importance was used to keep comparisons consistent and easy to interpret. The implementation of permutation importance means each feature in the test set is randomly shuffled and the drop in performance (Weighted-F1) is measured. Features that cause a larger performance drop are considered more important.

3. Results

The accompanying implementation (script_EmmaGogic.py) generates:

- metrics_table.csv - compares all models on the same holdout test split
- classification_report_GradientBoosting.txt -
- confusion_matrix_GradientBoosting.csv
- permutation_importance_GradientBoosting.csv

The performance of all six classifiers on the stratified 80/20 holdout test set is summarized in the table below, with the overall best classifier highlighted:

Model	Accuracy	Macro-F1	Weighted-F1
Gradient Boosting	0.950	0.950	0.951
Random Forest	0.936	0.936	0.938
Decision Tree	0.917	0.916	0.918
Logistic Regression	0.875	0.871	0.873
KNN	0.842	0.825	0.829
Linear SVM	0.768	0.763	0.763

The table indicates that the Gradient Boosting classifier achieved the highest performance across all three metrics, with accuracy = 95.04%, Macro-F1 = 0.950, and Weighted-F1 = 0.951. In addition to having the highest accuracy, the Gradient Boosting classifier

maintained balanced performance across all seven obesity categories. The second-best model was the Random Forest classifier, with a slightly lower Weighted-F1. The results demonstrate that ensemble boosting methods provide the strongest predictive performance for this dataset.

4. Discussions

Tree ensembles (Random Forest, Gradient Boosting) tend to perform strongly on datasets such as this one, that contain different types of data and multiple categories. The fact that the gradient boosting classifier was determined to be the best-performing method shows that obesity categories are influenced by combinations of variables rather than by simple linear relationships. Furthermore, it indicates the complexity of the relationships between the variables in the dataset and explains why linear models such as Logistic Regression and Linear SVM may underfit when mapping from habits to obesity category.

Permutation importance revealed that weight- and activity-related variables were among the most influential predictors. However, there are some limitations in the data collection that should be noted.

A major limitation of this dataset is that a large portion of the data (77%) is synthetically generated. Although synthetic data can often make the classification problem easier than real-world data, it may inflate test performance if the patterns in the synthetic data are stronger than those in the real data. Another limitation is relying on a single holdout split by default; in order to reduce variance and provide a more reliable estimate, cross-validation should also be used.

Finally, the emergence of unexpected errors in the code required debugging and gave valuable insights into practical workflow challenges in data mining. For instance, the Logistic Regression parameter error illustrated how machine learning libraries evolve over time, so code that works in one version may not work in another. Additionally, the median-imputation error highlighted the importance of robust data type validation before preprocessing. Improper feature classification can silently introduce bias or cause crashes. Implementing a coercion-based detection strategy ensured proper separation of numeric and categorical attributes, strengthening the reliability of the preprocessing pipeline.

5. Conclusions

The aim of this project was to investigate the link between obesity levels, and eating habits and physical condition. This was achieved in Python using data mining techniques such as classification. After thorough preprocessing of data including imputation, scaling, and one-hot encoding, six classifiers were compared using stratified evaluation and several metrics. Based on Weighted-F1, the gradient boosting classifier was determined to be the best-performing model. The results further demonstrate the

differences between model families including the ways in which they interpret data and how they handle bias/variance in a dataset.

References

[1] Estimation of Obesity Levels Based On Eating Habits and Physical Condition [Dataset].

(2019). UCI Machine Learning Repository. <https://doi.org/10.24432/C5H31Z>.